ReduLink: Authenticated Reference Substitution for Redundancy-Suppressed Transfers over Encrypted QUIC Streams

Michél Nguyen
University of the People | ORCID: 0000-0001-6834-4422

Abstract

Encrypted transports such as QUIC deliberately hide payload bytes from the network, which disables the transparent in-network redundancy elimination that wide-area optimizers have historically relied on. ReduLink is an endpoint-controlled representation layer that recovers redundancy savings for receivers that already hold related bytes, without breaking encryption: it replaces repeated payload chunks with compact references that are individually authenticated against epoch, scope, stream identifier, reconstructed offset, length, nonce, and chunk identity. The contribution is not a new chunk-matching algorithm and not a claim of faster physical links; it is authenticated, scoped, QUIC-compatible reference substitution with explicit reconstruction, privacy scoping, and fail-closed semantics.

The artifact maps bounded compact-binary ReduLink messages into server-authenticated aioquic streams, implements semantic MISS/FULL repair, derives a fresh per-run record key from an exporter surrogate plus random connection context, and independently validates receive sequence and reconstructed offset. It does not expose live QUIC TLS exporter bytes and does not authenticate the client. Exact object experiments reconstruct the ordered mapping of names, boundaries, empty objects, and contents; the secure profile also authenticates object headers. We compare fixed reuse, gzip/zstd, rsync, and a pinned zstd dictionary-delta baseline across deterministic and hash-pinned public bytes, and report local QUIC loss, scale, miss-rate, and path-emulation measurements. The legacy Linux tc/netem run launched raw and ReduLink transfers concurrently, so it is retained only as a contention diagnostic, not isolated single-flow latency or fairness evidence.

On object-aligned public releases the corrected model reaches 1.92x to 9.31x (1.82x to 7.65x at the 29-byte-scope wire profile), and 21.37x (15.28x wire-priced) on the layer-like case. Baselines delimit the claim: rsync dominates ordinary source-tree updates, while zstd 1.5.7 --patch-from reaches about 66x to 1,801x where an exact prior stream and codec delta are acceptable. This baseline is not RFC 9842 Compression Dictionary Transport; RFC 9842 is an HTTP content-coding protocol with dictionary hashes and origin/availability constraints. QUIC AEAD already protects the on-path channel. ReduLink's additional HMACs instead bind post-TLS reference and dictionary state to epoch, scope, stream, offset, nonce, identifier, and length. The supported result is therefore conditional byte reduction with byte-exact, fail-closed reconstruction, not universal compression, lower latency, or Internet fairness.

Keywords: QUIC; HTTP/3; redundancy elimination; data deduplication; content-defined chunking; shared dictionaries; authenticated references; encrypted transport; reproducible evaluation.

1. Introduction

Encrypted transports restrict transparent in-network redundancy elimination. QUIC combines UDP transport, TLS security, stream multiplexing, loss recovery, and congestion control, which makes middlebox rewriting of payloads the wrong abstraction for most public or end-to-end encrypted deployments [6-10]. A WAN optimizer can only suppress redundancy that it can see, and a correctly deployed QUIC connection ensures that it sees only ciphertext. The redundancy, however, has not disappeared: registries, content-delivery networks, software-update services, and backup systems still send byte-stable objects to receivers that already hold closely related bytes from a previous transfer.

The practical question is therefore not whether repeated bytes can be replaced by references. Fixed-block reuse, rsync, and low-bandwidth file systems already demonstrate that repeated bytes can be avoided [1-5], and HTTP delta and shared-dictionary mechanisms show that receiver-held bytes can be reused as a dictionary on the web [19-22]. The question this paper addresses is whether a reference-substitution mechanism can be made explicit, individually authenticated, privacy-scoped, and compatible with encrypted endpoint streams, without claiming a universal accelerator and without asking the application to become a file-tree synchronization protocol. Figure 1 shows the

resulting architecture: cooperating endpoints that already control the plaintext decide, per chunk, whether to send bytes or an authenticated reference, and the receiver reconstructs or fails closed.

We use one accounting vocabulary throughout. The effective multiplier is reconstructed or input bytes divided by encoded bytes at a stated accounting layer. Stream-payload multipliers exclude UDP/IP/link overhead; UDP-estimated multipliers add a local IPv4/UDP estimate; and congestion fairness, where claimed, applies to encoded bytes, not reconstructed bytes. This separation is deliberate: ReduLink changes how many application bytes a given number of wire bytes can reconstruct, not the physical line rate.

This paper investigates three questions. RQ1: can reference substitution over encrypted QUIC streams be made explicit, authenticated, and privacy-scoped without custom transport-layer changes? RQ2: under which workload shapes does authenticated reference substitution save bytes, and where do simpler fixed-block reuse, compression, or rsync win? RQ3: what is the component-level cost of the authentication and accounting machinery, and how stable and fair are the native QUIC results across repeated runs, scales, and emulated bottlenecks? Section 7 answers RQ2, Sections 8 and 10 answer RQ3, and Sections 4 and 5 answer RQ1.

2. Contributions and Claim Boundary

- An authenticated ReduLink reference format and validation model for endpoint-controlled encrypted streams, with per-frame binding of epoch, scope, stream id, reconstructed offset, length, nonce, and chunk identity.
- A compact binary mapping carried inside native aioquic QUIC streams, with byte-exact semantic MISS/FULL repair, repeated trials, and a payload-scaling experiment.
- A server-authenticated native path with fresh per-run exporter-surrogate key input, random connection context, independent receive-state checks, and negative tests for wrong secret, scope, epoch, stream, offset, length, replay, and tampering.
- A comparison against fixed-block reuse, gzip/zstd, and real rsync, including cases where these baselines win decisively, plus an explicit contrast with HTTP delta and shared-dictionary transport.
- External public source-release negative evidence, object-aligned public source-release workloads, and a public Redis-derived layer-like positive case, all hash-pinned and reproducible.
- Block-size sensitivity, component-cost measurements, paired local path-emulation diagnostics, and conservative accounting-layer separation without a production-fairness claim.

The central claim is conditional. ReduLink helps when byte-stable chunks recur across warm endpoint state and authenticated reference substitution is preferable to a file-oriented delta protocol or local compression. It is not a replacement for rsync or compression, and the artifact is a native QUIC stream mapping rather than a custom QUIC extension-frame implementation. A practical example is a registry, CDN, backup, or update service that repeatedly sends object records to the same endpoint or tenant while preserving end-to-end QUIC encryption. In that setting the server can reference receiver-known objects or chunks without exposing plaintext to a middlebox; the price is authentication and metadata overhead, and the benefit is explicit context binding, safe miss repair, and a stream-compatible representation.

3. Background and Related Work

3.1 Network redundancy elimination

Spring and Wetherall introduced protocol-independent redundancy elimination, suppressing repeated byte ranges on a link by caching recently seen content at both ends [1]. Subsequent work generalized this to network-wide and coordinated redundancy elimination (SmartRE) and to enterprise and end-system services (EndRE), and measured how much redundancy real traffic contains [2-4]. These systems are powerful but assume a vantage point that can observe or terminate plaintext. ReduLink targets the opposite regime, where the transport is end-to-end encrypted and only the endpoints can act, so the redundancy decision must move into the endpoint stack.

3.2 File-delta and low-bandwidth file systems

rsync and the Low-Bandwidth File System (LBFS) show the value of transferring only the differences between file objects that both sides can name and compare [5]. They are the right tool when the workload is genuinely a file tree and both endpoints can run a delta protocol. As our negative results in Section 7.1 confirm, ordinary source-tree updates are better served by rsync than by stream-level reference substitution. ReduLink is not aimed at this case; it is aimed at object or stream delivery where a file-tree delta negotiation is not the serving abstraction.

3.3 HTTP delta and shared-dictionary transport

The closest deployed lineage is HTTP receiver-side reuse. RFC 3229 defines delta encoding relative to a prior response, commonly with VCDIFF [19,20]. The 2016 SDCH proposal described shared dictionaries across HTTP responses [21]. RFC 9842 Compression Dictionary Transport lets an HTTPS origin advertise a previously fetched response for Brotli or Zstandard dictionary use [22]. Its dictionary identity is a SHA-256 hash; availability and same-origin constraints govern selection; and decoding failure causes the affected response to be discarded. It therefore has explicit integrity and failure semantics and can operate over HTTP/3 on QUIC.

ReduLink is not a more secure replacement for RFC 9842. QUIC TLS/AEAD already protects either mechanism against on-path modification, while RFC 9842 validates dictionary identity. ReduLink instead offers a different representation granularity: individually resolved chunk references bound to epoch, scope, stream, reconstructed offset, length, and nonce, with an explicit MISS then FULL state-repair exchange outside HTTP content coding. The per-record HMAC is useful at the post-TLS representation boundary, where implementation bugs, stale or shared dictionary state, and cross-context confusion must fail closed; it is not claimed as a second independent network-adversary barrier. Section 12 therefore compares deployment abstractions, and the zstd --patch-from experiment is labeled a whole-stream dictionary-delta byte baseline rather than an implementation or analog of RFC 9842.

3.4 Content-defined chunking and deduplication privacy

Content-defined chunking determines where chunk boundaries fall and strongly affects deduplication ratio and throughput; recent surveys and fast designs analyze the trade-offs in depth [11,12]. Chunking and deduplication also create content-existence side channels: deduplication can leak whether a receiver already holds particular content, and chunk-boundary parameters can themselves be attacked [13,16,17]. These results directly motivate ReduLink's privacy scoping (Section 4) and its exclusion of global cross-user dictionaries. Modern Ethernet line rates continue to increase [14], which is why the paper avoids physical-rate claims and reports only representation-layer savings.

3.5 Deployment model

The deployment model has three roles: a sender with access to the new byte stream, a receiver with a scoped warm dictionary, and a policy domain that defines whether dictionary state is per connection, per origin, or per tenant. Public Internet mode defaults to per-connection dictionaries. Public artifact mode may use signed or manifest-controlled per-origin dictionaries. Enterprise mode may use per-tenant dictionaries with audit and quota controls. Global private cross-user dictionaries remain out of scope. Table 1 summarizes when ReduLink is and is not the right tool relative to rsync and compression.

Table 1. ReduLink deployment modes and dictionary scoping.

Deployment	Dictionary scope	Why ReduLink rather than rsync/compression
CDN or registry objects	Per origin or per client	Objects arrive as encrypted streams; file-tree delta negotiation may not be the serving abstraction.
Backup/page streams	Per endpoint or tenant	Page-like chunks recur across snapshots and can be referenced while preserving authenticated reconstruction.
Enterprise VPN/admin domain	Per tenant	Policy can allow shared warm state while keeping middleboxes away from plaintext.
Source-tree sync	File tree	Usually better served by rsync; ReduLink is not targeted here.

4. Protocol and Security Model

ReduLink is a representation layer for cooperating authorized endpoints. QUIC packetization, packet numbers, ACK processing, TLS, and congestion control remain QUIC responsibilities. The native artifact verifies the server's ephemeral certificate against its local trust anchor; it uses no client certificate and therefore must not be described as mutually authenticated. ReduLink defines how application bytes are represented as authenticated FULL, REF, MISS, and DICT_ACK records and how the receiver validates them. A REF is deliverable only if dictionary membership and content, epoch, scope, stream id, independently expected reconstructed offset, length, nonce, chunk identity, and authentication tag all validate; otherwise the receiver fails closed and requests semantic FULL repair.

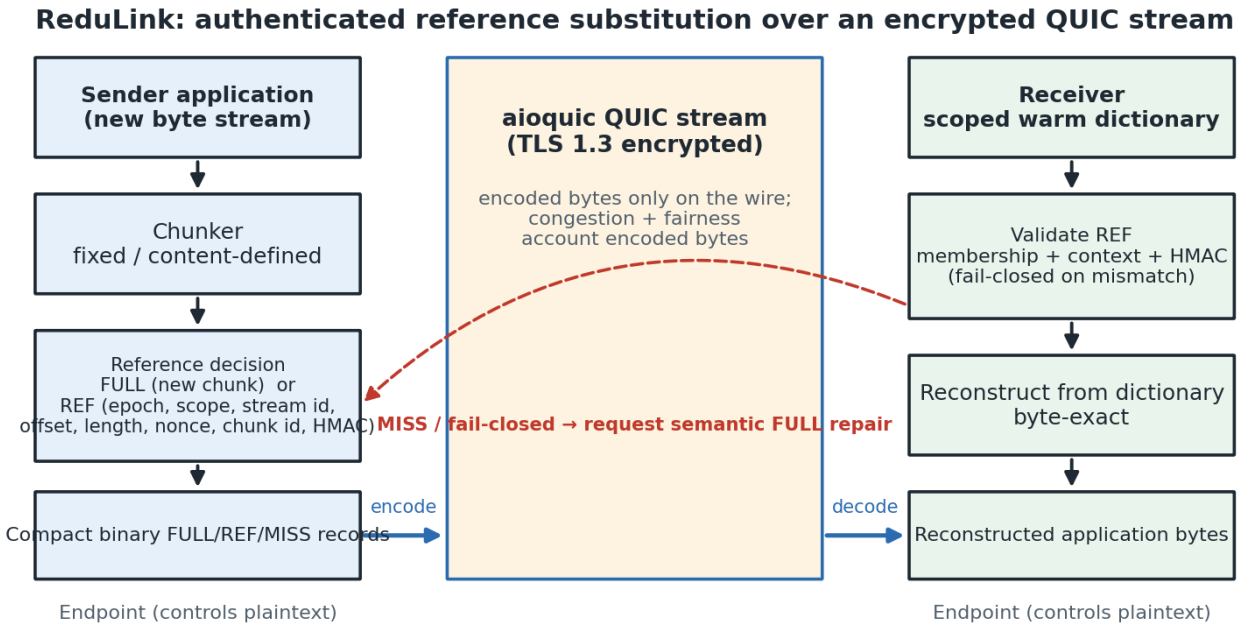

Figure 1. ReduLink protocol and deployment architecture. The sender chunks outgoing bytes and emits FULL or authenticated REF records carried as compact binary application data inside an encrypted QUIC stream; the receiver validates each reference against dictionary membership and bound context before byte-exact reconstruction, failing closed and requesting semantic FULL repair on any mismatch.

4.1 Frame types and wire model

ReduLink uses four logical frame types. A FULL carries and admits new bytes; a REF names a receiver-held chunk; MISS requests FULL fallback; and DICT_ACK may acknowledge admission. The stream_offset denotes reconstructed application position, never encoded-frame position. The receiver derives the expected offset from its own accepted state; it never validates an offset against the frame's own value. The offline model charges 24 bytes per FULL and 32 per REF. The compact binary artifact instead has a fixed 79-byte per-frame cost plus the UTF-8 scope length: 4-byte message length, 1-byte type, and 74 bytes of fixed fields (including identifier and tag). Thus the native 16-byte scope costs 95 bytes, while the 29-byte repricing profile costs 108. Section 7.6 states the selected profile rather than treating 108 bytes as universal.

Table 2. ReduLink frame types and their authenticated fields.

Frame	Purpose	Bound / carried fields
FULL	Carry and admit new chunk bytes	epoch, stream id, stream offset, chunk length, chunk id, payload, auth tag
REF	Reference a chunk already at the receiver	epoch, stream id, stream offset, original length, chunk id, reference nonce, auth tag
MISS	Request FULL fallback (fail-closed)	epoch, stream id, stream offset, chunk id
DICT_ACK	Acknowledge dictionary admission	epoch, chunk id, dictionary generation

4.2 Validation and fail-closed reconstruction

The receiver applies a fixed validation order before any byte is delivered, summarized as follows:

- Reject frames whose epoch does not match the active epoch.
- For FULL, validate the authentication tag and chunk identifier, admit the payload to the dictionary, and deliver the bytes at the intended stream offset.
- For REF, validate the authentication tag, expansion bound, stream offset, original length, nonce, and dictionary presence; deliver reconstructed bytes only after all checks succeed.
- Emit MISS when the referenced chunk is unavailable or policy refuses the reference; a REF that cannot become valid in the current epoch triggers an immediate MISS.
- Apply stream ordering and flow control to reconstructed bytes, while applying congestion accounting to transmitted wire bytes.

A reference miss is treated as a synchronization failure, not a corruption event: the receiver delivers no reconstructed bytes, emits MISS, and the sender repairs with a FULL for the same stream id, offset, length, and chunk id. A REF whose original_length disagrees with the stored chunk length, or that would expand beyond the negotiated bound, is a hard validation failure. Section 11 reports prototypes that exercise exactly these miss, tamper, and replay paths.

4.3 Dictionaries, epochs, and key schedule

Dictionaries are per connection by default and per origin or tenant only under explicit policy; global private cross-user dictionaries are out of scope. Bounded LRU eviction, quotas, and epochs limit state. Chunk identifiers are keyed MACs over epoch, scope, and the chunk digest; frame tags additionally bind stream, offset, length, and nonce. aioquic's public API does not expose TLS exporter bytes, so the native experiment uses a fresh private random exporter surrogate and independent random connection context for every run, then applies the HKDF schedule over ALPN, epoch, scope, connection, and stream context. Only hashes of the public connection context are recorded. The standalone secure and UDP models retain fixed test keys for deterministic testing. A production profile must replace the surrogate with actual QUIC TLS exporter output.

4.4 Threat model

Table 3 separates the specified security design from what the artifact implements and tests. The artifact validates the reference-authentication design and its negative cases; it should not be read as production-grade cryptographic integration. It implements HMAC binding, chunk-id validation, nonce rejection, length and expansion checks, and fail-closed repair, but it does not implement production exporter-derived keys, custom extension-frame parsing, production replay windows, or cross-tenant isolation enforcement. Like other deduplication systems, ReduLink can create a content-existence oracle when an adversary can choose payloads or references and observe transfer size, timing, or MISS behavior; per-connection dictionaries remove the cross-user oracle in public mode, and shared dictionaries are permitted only for public artifacts or explicit trust domains. The security evidence in this paper is artifact-level functional testing plus the reduction-style analysis of Section 4.5; no side-channel mitigations (padding, constant-shape errors, timing equalization) are implemented in the artifact. Dictionary eviction is itself an oracle in shared-dictionary modes: an adversary who can insert FULL traffic can cascade-evict a victim's warm entries (Section 8.2 demonstrates the mechanics under the LRU budget) and then probe REF-versus-MISS behavior to infer content existence, which is why per-connection scoping remains the default and shared dictionaries require explicit policy.

Table 3. Security feature status: specified, implemented, tested, and future work.

Feature	Specified	Implemented	Tested	Future work
Per-reference integrity	FULL/REF tag binds chunk, length, stream offset, epoch, scope, nonce	HMAC-style tag and chunk-id checks	Tamper, wrong-context, wrong-length, reconstruction tests	Production key management
Context binding and replay	Exporter-derived keys, epoch/scope/stream/offset binding, replay window	HKDF-style schedule in aioquic path; fixed-secret model paths bind context in MAC message	Wrong scope/epoch/stream/offset and nonce replay rejection tests	Live QUIC TLS exporter bytes and production replay window
Dictionary safety	Authenticated FULL admission, manifest commitment, eviction policy	FULL chunk-id checks and bounded dictionary behavior	Budget overflow/recovery and fail-safe reconstruction tests	Signed-manifest policy and cross-tenant admission enforcement
Expansion and repair bounds	Per-frame length, per-stream caps, MISS then FULL repair	Length checks and semantic repair prototypes	MISS/FULL, UDP repair, authenticated-UDP negative probes	Full production flow-control integration

Feature	Specified	Implemented	Tested	Future work
Privacy scope	Per-connection default; origin/tenant sharing only by explicit policy	Policy modeled and documented	Negative context-binding tests	Side-channel padding/timing mitigation and tenant isolation

Table 4. Privacy modes and leakage risks.

Mode	Dictionary scope	Leakage risk	Default
Public Internet	Per connection only	Same-connection access-pattern leakage	Yes
Public artifacts	Per-origin signed manifest	Artifact-version / popularity inference	Optional
Enterprise VPN	Tenant / admin domain	Intra-tenant content-existence leakage	Optional
CDN/update channel	Origin-scoped public versions	Version-possession inference	Optional
Global cross-user	Any user	Private content-possession leakage	No (out of scope)

4.5 Formal security model and analysis

We separate two boundaries. A network adversary may observe, drop, reorder, inject, duplicate, or replay QUIC packets, but QUIC TLS/AEAD already prevents that adversary from creating or modifying accepted stream plaintext [6,7]; ReduLink does not claim an additional independent on-path guarantee. The ReduLink analysis instead models an adversary at the decoded-record boundary who can present chosen records, replay previously authenticated records, and, in policy-authorized shared modes, influence dictionary contents, but does not know the per-connection record key K. Production endpoints derive K from the QUIC TLS exporter via HKDF [24]. The artifact tests this interface with a fresh per-run surrogate on its native path and fixed keys in deterministic model paths.

We require four properties. Reference unforgeability: no efficient adversary can produce a FULL or REF frame that the receiver accepts and that delivers bytes the sender did not authenticate for the bound context, except with negligible probability. Context binding and replay resistance: a frame authenticated for a given epoch, scope, stream, and offset is rejected in any other context, and a nonce already seen within the epoch window is rejected. Expansion bound: an accepted REF delivers exactly the authenticated chunk length, so a small reference cannot reconstruct unbounded output. Dictionary safety: only authenticated FULL chunks, or signed-manifest chunks, are admitted to the dictionary. The theorem below covers the first two properties, with the replay and expansion bounds following from the same verification checks; manifest-mode dictionary safety is a specification requirement that is not analyzed here because the manifest mechanism is unimplemented (Table 3).

Concretely (Section 4.1), each chunk identifier is a truncated keyed MAC, cid = HMAC_K("cid", epoch, scope, SHA-256(chunk)), and each frame carries tag = HMAC_K(kind, epoch, scope, stream, offset, length, nonce, cid, SHA-256(payload)) [23]. Verification is authentication-first: the receiver recomputes the tag over the frame's own fields and rejects on mismatch with a single generic error, so an attacker without the key cannot distinguish which field was tampered with; only authentically-tagged frames proceed to the context checks (epoch, scope, stream, offset) and then to replay rejection against a bounded, reorder-tolerant nonce window. In the artifact the window holds the most recent nonces and treats anything at or below its floor as replayed, bounding receiver memory for long-lived sessions.

Theorem (informal). Model HMAC-SHA-256 as a pseudorandom function (PRF), its 128-bit truncation as a random 128-bit function up to PRF distinguishing advantage, and SHA-256 as collision resistant. Then, after q verification attempts and n distinct identifiers in one key domain, acceptance of a fresh unauthorized frame is bounded by the PRF advantage plus approximately q/2^128 for a tag guess; resolving an authenticated identifier to different bytes is additionally bounded by the SHA-256 collision advantage plus approximately n(n-1)/2^129 for a truncated-identifier collision. EUF-CMA alone supports fresh-message tag unforgeability but does not by itself justify the birthday estimate for collisions between truncated HMAC outputs, hence the explicit PRF/random-function assumption.

Proof sketch. A fresh accepted frame with no sender-authenticated canonical input either distinguishes HMAC from a random function or guesses its 128-bit tag. If a valid REF resolves to different bytes, the receiver's new content revalidation requires either a SHA-256 collision or equal 128-bit PRF outputs for distinct digests. Context fields are inside the tag and also checked against independently maintained receive state, so authentic cross-context records fail

after authentication; repeated or below-window nonces fail replay state. The native path has per-run key/context freshness, while a whole deterministic standalone-model session can be replayed to a new decoder that deliberately reuses its fixed test key. Expansion is bounded because the receiver compares the stored chunk length and reconstructed object boundary before delivery. These are reduction-style arguments, not a machine-checked proof.

The MAC input is canonical JSON with sorted keys and fixed separators; an interoperating implementation must reproduce this encoding or specify another canonical form. Both identifiers and tags are 128 bits in the artifact. Identifier sizing must therefore consider the total number of entries per key domain; very large or multi-tenant deployments should prefer 192 or 256 bits. The secure-model, native-state, wire-boundary, and authenticated-UDP tests exercise the stated checks, but production exporter integration, key lifecycle, and side-channel defenses remain future work.

5. Implementation and Metrics

The artifact is implemented in Python for reproducibility and uses aioquic for native QUIC stream experiments [18]. ReduLink messages are compact binary application-stream records inside QUIC streams; this exercises a real QUIC handshake and encryption but does not implement custom QUIC frame types or transport parameters. The JSON encoding is retained only as a readable baseline, and the default path uses the compact binary format, which encodes frame type, stream id, reconstructed offset, length, chunk id, nonce, authentication tag, and payload only where required.

Three byte layers are reported. Input bytes are the reconstructed application bytes. Stream-payload bytes are the ReduLink or raw application bytes written into QUIC streams. UDP-estimated bytes add a local IPv4/UDP estimate from the loopback proxy path. The paper avoids treating stream-payload multipliers as full wire-rate measurements; they isolate the representation layer so that raw QUIC, ReduLink, gzip, zstd, fixed reuse, and rsync-style baselines can be compared consistently. Table 5 records, for each evidence layer, what it supports and what it does not prove.

Two reproducibility notes apply. Byte multipliers are deterministic functions of bytes and chunking parameters. Wall-clock and throughput values are local diagnostics. Component metadata records Python 3.13.0b2 on an arm64 development Mac; some historical transport evidence was produced in a Linux namespace and is labeled separately. Timings should be read as within-host order-of-magnitude costs, not portable guarantees.

Table 5. Evidence hierarchy: what each layer supports and does not prove.

Evidence	Supports	Does not prove
Model	FULL/REF accounting and reconstruction	Transport behavior
Secure model	HMAC binding and replay checks	Live QUIC exporter use
aioquic stream	Encrypted QUIC stream mapping	Custom extension frames
UDP/IPv4 estimate	Local datagram accounting	Full packet capture
Workloads	Workload sensitivity	Universal acceleration
Path emulation	Measured shared-bottleneck completion and queueing (userspace)	Kernel netem or Internet-path fairness

6. Evaluation Methodology

Evaluation uses four classes of evidence. First, deterministic journal fixtures (disk snapshot, OCI layer, package metadata, repository snapshot, structured logs, and a compressed negative control) isolate predicted positive and negative workload shapes. Second, public source-release pairs (Click, Redis, nginx) test ordinary source-tree updates where ReduLink should not be assumed to help. Third, object-aligned public release experiments use the same public bytes but model registry/CDN object delivery rather than tarball synchronization. Fourth, native aioquic stream experiments measure the encrypted stream mapping on positive and negative cases, under loss, at scale, and against competing flows and emulated bottlenecks. The journal fixtures are constructed with a chosen unchanged fraction and therefore illustrate workload shapes rather than estimate real-world gains; the only fully external inputs are the public source releases, which are negative for ReduLink, and their object-aligned re-framing, which is the paper's primary external positive signal. An author-independent-overlap version-pair study of real package upgrades is added in

Section 7.5, and Section 7.6 re-prices all headline results at the measured wire framing and adds a dictionary-delta baseline.

For each workload the paper reports ReduLink fixed chunking, ReduLink content-defined chunking where relevant, fixed-block reuse, compression, and rsync where the baseline is structurally applicable. The fixed-block baseline is intentionally strong and simple: if it beats ReduLink, the result is reported rather than hidden, because ReduLink is a security and transport-compatibility layer over reuse, not a superior matching algorithm. All external corpora are hash-pinned, and every figure and table is regenerated from committed result files.

7. Workload Results and Baseline Interpretation

Table 6 reports the deterministic warm-state fixtures. On most byte-stable workloads, fixed-block reuse matches or beats ReduLink; the distinct ReduLink contribution is not superior matching but authenticated, scoped, stream-compatible reference substitution with explicit repair and privacy rules. The negative rows (package metadata, logs, and the compressed control) are included precisely because they fail: where repetition is local to one object, compression wins, and where there is no reusable structure, both fixed reuse and ReduLink correctly decline to help. The Unchanged column makes the mechanism explicit: because for aligned whole-chunk reuse the effective multiplier is bounded by roughly one over one minus the unchanged fraction (the fixed-block-reuse column can exceed this bound because its byte-scan alignment also matches moved content), these deterministic fixtures (96 to 100 percent unchanged on the positive rows) are illustrative workload shapes chosen to expose where reference substitution can and cannot help, not estimates of real-world overlap. The repository fixture in particular changes only 210 of 501,760 bytes, so its 24.73x is close to a no-op transfer.

Table 6. Warm-state workload results (effective stream-payload multiplier), with the constructed unchanged fraction of each fixture.

Workload	Unchanged	ReduLink	Fixed reuse	Best compression	RL - fixed
disk snap	97.3%	28.49x	31.97x	1.06x	-3.48x
oci layer	96.5%	11.05x	10.68x	1.50x	+0.37x
package meta	3.5%	0.99x	9.88x	14.31x	-8.89x
repository snap	100.0%	24.73x	24.60x	10.37x	+0.14x
logs	3.3%	0.99x	14.89x	18.01x	-13.89x
compressed neg	0.4%	0.99x	1.00x	1.00x	-0.01x

7.1 External Public Source Releases

Table 7 reports three independent, hash-pinned source-release pairs. The result is negative for ReduLink and strongly positive for rsync. This is an important and deliberately reported finding: ordinary related source trees are better served by file-oriented delta transfer than by stream-level reference substitution, because rsync can exploit fine-grained intra-file similarity that whole-object referencing cannot.

Table 7. External public source-release pairs: ReduLink and fixed reuse versus real rsync (negative case for ReduLink).

Public pair	ReduLink	Fixed reuse	rsync total
click-8.1.7 to 8.1.8	0.99x	0.99x	6.48x
redis-7.2.4 to 7.2.5	1.00x	1.00x	73.13x
nginx-1.25.3 to 1.25.4	1.00x	0.99x	49.49x

7.2 Object-Aligned Public Release Transfers

Table 8 and Figure 2 use the same public tarballs but change the transfer abstraction: every relative name, object length, empty object, and content byte is encoded and decoded exactly, while chunking restarts at object boundaries and warm state remains shared across objects. The secure profile MACs each name/length boundary and binds frames to an object-derived stream id. Reconstruction success is computed by comparing the complete ordered object mapping, not assigned by accounting logic. Redis and nginx remain positive because many objects are stable; Click is weaker. This is external transfer-model evidence, not a captured registry trace.

Table 8. Object-aligned public release transfers modeling registry/CDN object delivery.

Public release	File stability	Bytes	ReduLink	Secure RL	Fixed object reuse	gzip
click	83/147 unchanged	947,826	1.92x	1.88x	1.92x	2.84x
redis	1558/1583 unchanged	15,467,095	9.31x	8.58x	9.30x	4.59x
nginx	468/503 unchanged	7,357,392	4.33x	4.18x	4.32x	6.07x

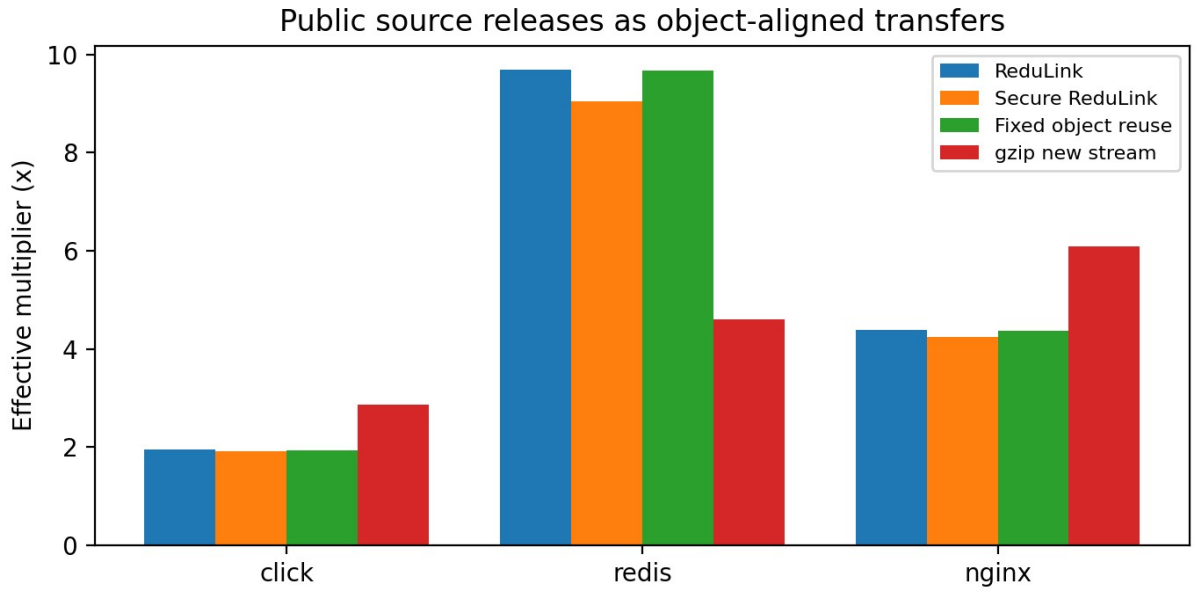

Figure 2. Object-aligned public release transfers: effective stream-payload multipliers for ReduLink, the authenticated variant, fixed object reuse, and gzip.

7.3 Layer-Like External-Positive Case

Table 9 reports a Redis-derived layer-like case built from included public Redis release bytes and aligned changed blocks. Its role is to test the predicted positive case for layer-like objects where stable chunks remain aligned; it is not a production trace. Here ReduLink reaches 21.37x with the authenticated variant at 18.96x, again close to the fixed-reuse reference.

Table 9. Layer-like external-positive case derived from public Redis release bytes.

Case	Input bytes	ReduLink	Secure RL	Fixed reuse
redis-layered-public-positive	524,288	21.37x	18.96x	21.33x

7.4 Block-Size Sensitivity

Table 10 and Figure 3 show that the best block size is workload-dependent: smaller blocks capture more aligned reuse but pay more per-chunk overhead, while larger blocks lose alignment. The artifact therefore treats 4 KiB as a reproducible default rather than a universal optimum, and a production deployment would tune the chunk size per workload class. The content-defined chunker consistently underperforms fixed chunking on these aligned fixtures (at 4 KiB: 10.71x versus 28.49x on the disk snapshot, 4.60x versus 11.05x on the OCI layer), because the fixtures change bytes in place without insertions; CDC's insertion robustness is not exercised here, and its Python implementation is also the slowest component in Table 17.

Table 10. Block-size sensitivity of the effective multiplier (fixed chunking).

Workload	1 KiB	2 KiB	4 KiB	8 KiB	16 KiB
disk snapshot	17.13x	23.33x	28.49x	17.08x	8.99x
oci layer	12.11x	11.86x	11.05x	5.91x	3.60x
repository snapshot	20.21x	24.99x	24.73x	14.45x	7.55x

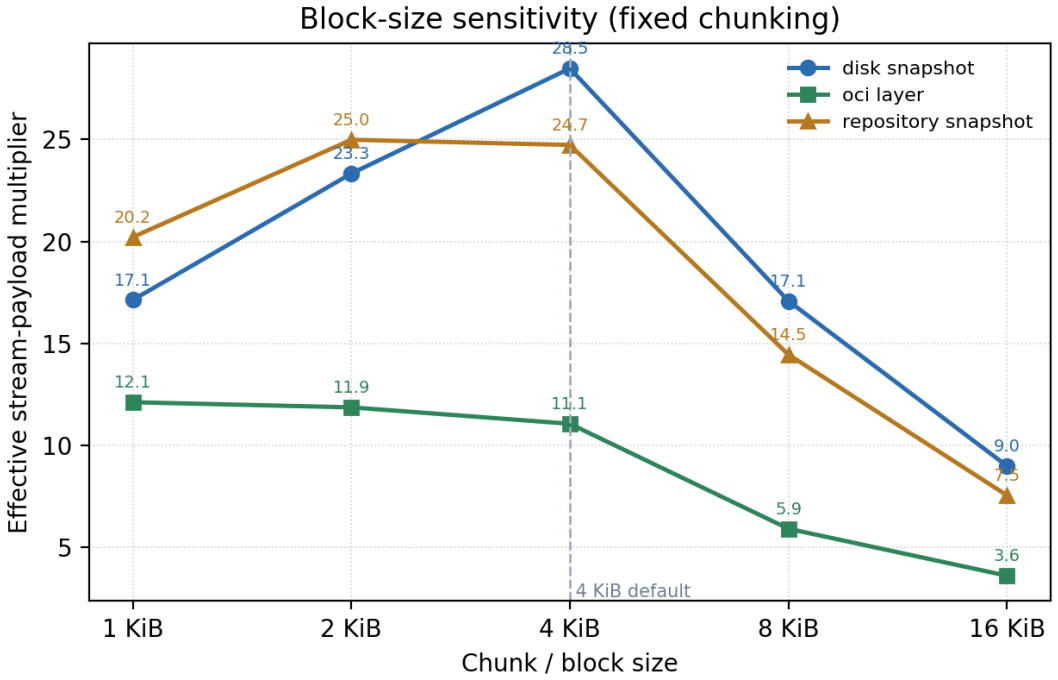

Figure 3. Block-size sensitivity of the effective multiplier for fixed chunking across three warm-update fixtures.

7.5 Package version-pair study (PyPI)

Table 11 applies the identical exact object encoder/decoder to six hash-pinned pairs of real PyPI wheels. The prior flat concatenation check has been removed because it used different boundaries and did not validate names. Patch releases with high file stability (rich, 77 of 83 objects unchanged; click, 14 of 22) reach about 11x to 12x, whereas jinja2, urllib3, and packaging change enough objects that gzip wins. The package/version selection remains author-chosen, so this is not population-weighted or a client trace. PyPI currently transfers compressed wheel archives rather than per-member objects; realizing these modeled gains would require a different registry serving abstraction. The gzip comparison is over the same uncompressed named-object stream and should not be mapped directly to today's wheel traffic.

Table 11. PyPI version-pair study: real package upgrades (object-aligned, hash-pinned).

Package	Versions	Files unch.	ReduLink	Secure RL	Fixed reuse	gzip	Recon.
rich	13.7.0->13.7.1	77/83	12.05x	10.92x	12.02x	4.42x	OK
jinja2	3.1.3->3.1.4	8/31	1.09x	1.08x	1.09x	4.02x	OK
click	8.1.6->8.1.7	14/22	10.93x	10.08x	10.90x	3.97x	OK

Package	Versions	Files unch.	ReduLink	Secure RL	Fixed reuse	gzip	Recon.
werkzeug	3.0.1->3.0.2	51/72	3.02x	2.94x	3.02x	3.95x	OK
urllib3	2.1.0->2.2.0	20/40	1.86x	1.83x	1.86x	3.80x	OK
packaging	23.2->24.0	8/21	1.53x	1.50x	1.52x	3.77x	OK

7.6 Framing sensitivity and a dictionary-delta baseline

Table 12 reports two review-driven checks. First, it re-prices model frames at the compact binary profile selected for this experiment: fixed 79-byte overhead plus a 29-byte scope, or 108 bytes per FULL/REF. This is profile-specific; the native 16-byte scope costs 95 bytes. With exact object headers included, Redis falls from 9.31x to 7.65x and the layer-like case remains 21.37x to 15.28x. Second, zstd 1.5.7 [26] uses the prior serialized object stream with the pinned command zstd -3 --patch-from OLD NEW -o PATCH -f -q. It reaches about 66x to 1,801x and dominates whenever the exact prior stream and codec delta are acceptable. This is a whole-stream dictionary-delta baseline, not RFC 9842: it does not exercise HTTP dictionary advertisement, SHA-256 dictionary identity, availability/same-origin rules, or response failure handling. ReduLink is therefore not byte-optimal; its distinct abstraction is incremental chunk resolution with authenticated post-TLS context and explicit semantic repair.

Table 12. Framing repricing (79 fixed bytes + 29-byte scope = 108 bytes) and pinned zstd 1.5.7 whole-stream dictionary delta on the same object streams.

Case	Input bytes	RL (model)	RL (wire-priced)	zstd --patch-from	gzip
click-8.1.7-to-8.1.8	947,826	1.92x	1.82x	66.51x	2.84x
redis-7.2.4-to-7.2.5	15,467,095	9.31x	7.65x	1800.59x	4.59x
nginx-1.25.3-to-1.25.4	7,357,392	4.33x	3.95x	1002.78x	6.07x
redis-layered-public-positive	524,288	21.37x	15.28x	1795.51x	4.24x
pypi rich-13.7.0-to-13.7.1	947,654	12.05x	9.49x	591.91x	4.42x
pypi jinja2-3.1.3-to-3.1.4	491,214	1.09x	1.07x	170.74x	4.02x
pypi click-8.1.6-to-8.1.7	353,767	10.93x	8.92x	631.73x	3.97x
pypi werkzeug-3.0.1-to-3.0.2	762,091	3.02x	2.83x	402.58x	3.95x
pypi urllib3-2.1.0-to-2.2.0	391,321	1.86x	1.78x	29.51x	3.80x
pypi packaging-23.2-to-24.0	174,172	1.53x	1.47x	81.35x	3.77x

8. Native QUIC Experiments

8.1 Stream mapping under loss

Table 13 compares raw and ReduLink stream mapping at zero and periodic datagram loss, and Table 14 reports positive and negative workload cases. Every run verifies the ephemeral server certificate, uses a fresh record-key surrogate/context, checks HELLO metadata, and reconstructs exactly. The compressed-negative control correctly yields no gain. The Redis-layered corpus reaches 21.37x in the offline model but about 3.83x over QUIC because the native experiment uses 1 KiB chunks, pays the 95-byte native framing profile (79 + 16-byte scope) rather than the model's 32-byte REF charge, and deliberately withholds every seventh dictionary chunk, forcing 72 payload-bearing repairs. Offline tables isolate representation accounting; QUIC tables are end-to-end stream evidence.

Table 13. Native aioquic stream mapping: raw versus ReduLink under zero and periodic datagram loss.

Method	Loss every	Stream x	UDP-est x	Recon.
Raw	0	1.00x	0.90x	OK
ReduLink	0	3.19x	2.60x	OK
Raw	9	1.00x	0.80x	OK
ReduLink	9	3.19x	2.34x	OK

Table 14. Native aioquic stream mapping across positive and negative workload cases.

Case	Input	Stream x	UDP-est x	Misses	Recon.
demo positive	98,304	3.19x	2.60x	13	OK
compressed negative	262,242	0.91x	0.85x	0	OK
Redis layered	524,288	3.83x	3.50x	72	OK

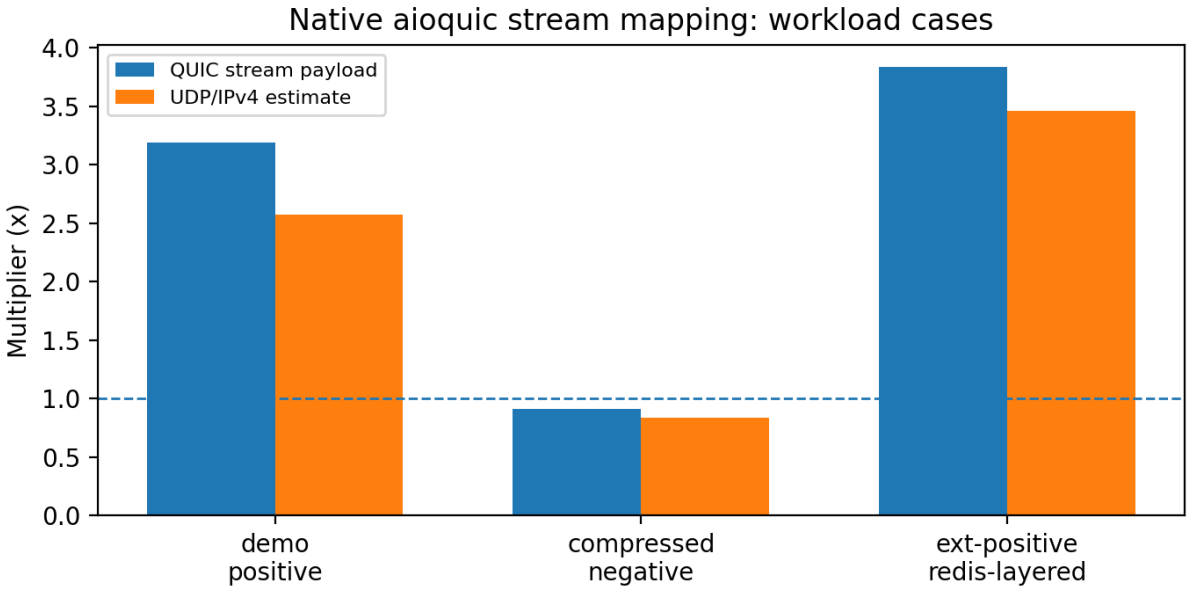

Figure 4. Native aioquic stream mapping on positive and negative workload cases.

8.2 Repeated trials and scaling

Table 15 reports 20 repeated trials of the native demo workload. The ReduLink stream multiplier is near-deterministic for fixed bytes, while elapsed times vary across localhost runs; reporting both avoids selecting one favorable timing. Table 16 scales from 96 to 16,384 blocks (96 KiB to 16 MiB). With the default 8,192-chunk budget, the 16,384-chunk warm set overflows the LRU; sequential FULL admission cascade-evicts remaining warm entries and degrades to a FULL-only 0.92x while still reconstructing exactly. Raising the budget to 24,576 restores 3.96x. Dictionary sizing relative to the warm working set is therefore a provisioning requirement; the single-run elapsed times are host-dependent diagnostics, not a speedup claim.

Table 15. Repeated native QUIC trials (n = 20): run-to-run stability of multipliers and elapsed time.

Metric	n	Mean	Std. dev.	Min	Max
raw client ms	20	103.185400	4.531822	99.378000	120.714000
redulink stream multiplier	20	3.190150	0.000038	3.190135	3.190238
redulink udp est multiplier	20	2.602875	0.003894	2.596102	2.608294
redulink client ms	20	101.877150	3.900543	99.065000	116.933000

Table 16. Payload scaling of the native aioquic stream mapping, including the dictionary-budget overflow and recovery at 16 MiB.

Blocks	Input bytes	Dict budget	Stream x	Misses	Client ms	Recon.
96	98,304	8,192	3.19x	13	103.1	OK
512	524,288	8,192	3.77x	73	123.7	OK
1024	1,048,576	8,192	3.87x	146	150.7	OK
4096	4,194,304	8,192	3.94x	585	373.1	OK
16384	16,777,216	8,192	0.92x	0	2235.4	OK
16384	16,777,216	24,576	3.96x	2340	1768.9	OK

9. Component Costs

Table 17 reports component-level throughput on the development Mac identified in results/component_performance.metadata.json. Fixed chunking, HMAC validation, and compact binary encoding are fast in this local run; the Python content-defined chunker is slow. A production implementation would require optimized native chunking and tighter transport integration. These are hardware-dependent diagnostics, not line-rate guarantees.

Table 17. Component-level cost measurements (local Python artifact timing; not production line-rate).

Component	Input bytes	MiB/s local	Roundtrip
fixed chunking	2,097,152	7274.9	not applicable
CDC chunking	2,097,152	8.3	not applicable
model fixed roundtrip	1,573,964	611.5	OK
model CDC roundtrip	1,573,964	3.3	OK
HMAC roundtrip	1,573,964	149.0	OK
HMAC decode	1,573,964	301.4	not applicable
wire encode	1,573,964	4590.4	not applicable
wire decode	1,573,964	2438.6	not applicable

10. Transport Accounting and Local Diagnostics

Scope note: Section 10 provides localhost accounting and emulation evidence, not transport-fairness proof. The userspace shaper has an explicit shared bottleneck; the unshaped concurrent diagnostic does not. The committed Linux tc/netem rows are a legacy v3.13 run in which raw and ReduLink were launched concurrently, so they cannot be interpreted as isolated single-flow latency. None of these results proves WAN performance, Mininet behavior, production congestion fairness, or deployment-population effects.

The central fairness rule is that ReduLink must not receive congestion credit for reconstructed bytes: congestion control and bottleneck service account encoded wire bytes only. Table 18 reports the wire-byte accounting experiment, in which ReduLink's encoded wire share is 0.083 against the raw stream's 0.917 over 16 rounds, while still reconstructing the same application bytes; the effective application multiplier on that 48 KiB wire-accounting fixture is 10.72x. This confirms that the savings appear as fewer encoded bytes, which is the only place a fair transport should credit them.

Table 18. Wire-byte fairness accounting: ReduLink is charged on encoded bytes, not reconstructed bytes.

Metric	Value
Fairness rule	Congestion and bottleneck service count encoded wire bytes
Raw encoded wire share	0.917
ReduLink encoded wire share	0.083
ReduLink uses less wire than raw	yes
Effective application multiplier	10.72x
Rounds	16

Table 19 reports 20 concurrent raw/ReduLink localhost rounds with periodic datagram loss. There is no controlled rate limit or bottleneck, and the rate hint is metadata only. The previous artifact computed one Jain value [25] across all observations and called it fairness; v3.14 instead computes a two-flow balance index within each round and reports the mean solely as a concurrency diagnostic. Because the methods intentionally encode different byte volumes, neither encoded-rate nor reconstructed-rate balance is a congestion-fairness measure. Table 20 adds a shared userspace bottleneck, Table 21 varies miss rate, Table 22 reports local repeat variability, and Table 24 is only a closed-form byte-volume illustration.

Table 19. Unshaped concurrent localhost diagnostic (mean of per-round two-flow balance indices; not fairness).

Metric	Value
Scenario	20 concurrent rounds, unshaped localhost (no rate limit), loss every 9
Encoded-rate balance index (mean)	0.842
Reconstructed-rate balance index (mean)	0.982
All flows reconstructed	yes
Scope	unshaped localhost concurrency diagnostic; no fairness inference

Table 20 adds a measured full-duplex userspace bottleneck: both flows share one token bucket per direction plus propagation delay, across 5/20 Mbps and 20/80 ms points with 20 rounds each. Completion comparison now uses the mean of within-round ReduLink/raw ratios; the ratio of aggregate means is retained in JSON only as a diagnostic. On the refreshed demo run the paired mean ranges from 0.87x to 1.01x; Redis ranges from 0.82x to 1.30x. Reconstruction is exact throughout. The variation supports only a conditional local result: byte savings may reduce completion time, while MISS/FULL round trips and scheduling may erase or reverse it. Identical NewReno stacks and encoded-byte accounting are necessary conditions for a fair deployment, but this harness does not prove Internet fairness.

Table 20. Measured full-duplex userspace shared-bottleneck emulation: mean completion time and mean paired completion ratio (20 local rounds per scenario).

Payload	Rate Mbps	RTT ms	Raw ms	ReduLink ms	Mean paired RL/raw	Rate balance diag.	Recon.
demo	5.0	20.0	1005.8±273.8	935.3±79.4	0.97x	0.788	OK
demo	5.0	80.0	1231.1±40.4	1067.7±32.3	0.87x	0.820	OK
demo	20.0	20.0	366.7±17.8	323.4±9.8	0.88x	0.815	OK
demo	20.0	80.0	789.4±11.0	798.8±8.6	1.01x	0.783	OK
redis	5.0	20.0	2040.9±58.1	1933.9±102.0	0.95x	0.756	OK
redis	5.0	80.0	3021.0±674.4	3873.6±629.1	1.30x	0.691	OK
redis	20.0	20.0	1103.8±95.3	1247.4±77.6	1.13x	0.718	OK
redis	20.0	80.0	1427.1±29.5	1175.0±21.6	0.82x	0.788	OK

Table 21 turns the qualitative miss-rate claim into a sensitivity check. It uses the same native aioquic stream mapping and full-duplex userspace shaper at the constrained 5 Mbps, 20 ms point, but varies receiver dictionary thinning on the byte-stable demo payload. As the semantic miss fraction rises from about 1 percent to about 49 percent, ReduLink's stream multiplier falls from 5.75x to 1.48x and the completion-time advantage narrows from roughly 0.64-0.74x to 0.85x of the raw flow. This does not replace a broader rate/RTT grid, and the first row has high raw-flow timing dispersion, but it directly supports the mechanism-level explanation: repair payloads and reverse MISS/FULL round trips progressively consume the byte-saving advantage while preserving byte-exact reconstruction.

Table 21. Native QUIC miss-rate sensitivity at 5 Mbps and 20 ms RTT on byte-stable demo bytes (three rounds per point).

Dictionary thinning	Miss fraction	Raw ms	ReduLink ms	RL/raw	Stream x	Recon.
0	1.1%	925.8±289.9	664.5±76.3	0.74x	5.75x	OK
16	6.7%	1284.5±3.3	825.5±5.7	0.64x	4.31x	OK
8	13.3%	1319.5±1.8	968.2±12.5	0.73x	3.31x	OK
4	26.7%	1662.4±11.7	1322.3±1.3	0.80x	2.27x	OK
2	48.9%	2018.9±11.1	1722.8±12.5	0.85x	1.48x	OK

Table 22 reports deterministic percentile-bootstrap intervals over repeated local measurements, using within-round raw/ReduLink ratios where available. These intervals describe variability in these particular localhost runs; they are not confidence intervals for WAN paths, machines, deployments, or a sampled population. Byte ratios are stable, while completion intervals show that repair and scheduling can move results around parity. The macOS dummynet path still times out on the development Mac, and Table 23 is retained with an explicit legacy-design warning.

Table 22. Within-run variability for repeated native QUIC measurements (deterministic bootstrap intervals; no population inference).

Result	n	Mean	95% CI	Source
Demo constrained completion ratio	20	0.973	[0.904, 1.040]	quic_emulated_path
Demo encoded-byte ratio	20	0.313	[0.313, 0.313]	quic_emulated_path
Redis constrained completion ratio	20	0.948	[0.929, 0.964]	quic_emulated_path_redis
Redis encoded-byte ratio	20	0.261	[0.261, 0.261]	quic_emulated_path_redis
Concurrent localhost completion ratio	20	0.801	[0.749, 0.855]	quic_competing_flows
Concurrent localhost encoded-byte ratio	20	0.313	[0.313, 0.313]	quic_competing_flows
Sequential UDP-estimated multiplier	20	2.603	[2.601, 2.604]	repeated_quic_trials

Table 23 preserves the v3.13 Linux tc/netem data for transparency, but the original runner launched raw and ReduLink transfers concurrently under one qdisc. The reported 1.08x to 1.60x demo and 1.30x to 1.36x Redis completion ratios therefore combine method cost with mutual contention and cannot estimate isolated single-flow latency or establish the proposed half-duplex causal mechanism. The old runner also did not capture host, command, tool-version, or active-qdisc snapshots; the only retained provenance is the rootless namespace context and source commit. The v3.14 runner fixes the design by defaulting to isolated, order-alternated pairs and recording command, platform, Python, aioquic, tc, commit, and qdisc state. Until that corrected sweep is rerun on Linux, Table 23 is exploratory contention evidence. Its byte counts and reconstruction checks remain factual for the observed runs.

Table 23. Legacy v3.13 concurrent Linux tc/netem contention diagnostic (not isolated latency; incomplete environment provenance).

Payload	Rate Mbps	RTT ms	RL/raw completion (95% CI)	Enc-byte ratio	n	Recon.
demo	5.0	20.0	1.60x [1.525, 1.674]	0.313	20	OK
demo	20.0	20.0	1.08x [1.059, 1.095]	0.313	20	OK
redis	5.0	20.0	1.30x [1.263, 1.341]	0.261	12	OK
redis	20.0	20.0	1.36x [1.289, 1.415]	0.261	12	OK

Table 24. Analytic fluid-model completion times and implied application rates from measured encoded stream bytes (raw versus ReduLink); computed, not measured.

Rate Mbps	RTT ms	Raw compl. ms (model)	RL compl. ms (model)	Raw app rate Mbps (model)	RL app rate Mbps (model)
10.0	10.0	113.3	59.3	6.9	13.3
10.0	50.0	153.3	99.3	5.1	7.9
25.0	10.0	51.3	29.7	15.3	26.5
25.0	50.0	91.3	69.7	8.6	11.3
100.0	10.0	20.3	14.9	38.7	52.7
100.0	50.0	60.3	54.9	13.0	14.3

The package includes reviewer-runnable macOS dummynet and Linux tc/netem harnesses. Table 23 is a completed but methodologically concurrent legacy run, while the corrected isolated Linux sweep remains to be executed. A full Mininet or multi-host congestion-control study with independent production-style flows remains future work.

11. Repair and Authentication Prototypes

Beyond the in-stream experiments, three prototypes exercise the fail-closed and authentication paths directly. Table 25 summarizes them. The semantic-repair demo forces a 25 percent dictionary-miss fraction and confirms that every miss is repaired with a FULL and that reconstruction is byte-exact. The localhost UDP repair experiment drops every seventh data datagram and uses timeout-based retransmission, again reconstructing fully after 15 semantic repairs and 6 retransmissions. The authenticated UDP experiment adds explicit negative probes: a tampered authentication tag and a replayed nonce are both rejected (1 tamper and 1 replay rejection) before normal authenticated repair traffic is accepted, demonstrating fail-closed behavior under active manipulation.

Table 25. Repair and authentication prototypes (localhost).

Prototype	Input bytes	Misses / repairs	Negative probes rejected	Recon.
Semantic repair demo	32,546	2 / 2	n/a (25% missing)	OK
UDP repair (drop every 7)	49,152	15 / 15	6 retransmits	OK
Authenticated UDP	65,536	12 / 12	1 tamper, 1 replay	OK

12. Why Not Just Compress or Use rsync?

Compression is best when repetition is local to one object; rsync is best when both endpoints share a file tree and can run a delta protocol; HTTP delta or shared-dictionary transport is best when the application is HTTP and a codec-level dictionary suffices; and whole-stream dictionary delta (zstd --patch-from, Section 7.6) is best on raw bytes whenever the receiver retains the exact prior stream - by one to three orders of magnitude on our corpora. The ReduLink use case is different: encrypted endpoint streams where dictionary state is already available at the receiver, the transfer is not naturally a file-tree synchronization session, and each reconstruction step must be individually authenticated and fail closed. The evaluation confirms the boundary. On source-release trees, rsync dominates (Table 7). On package metadata and logs, compression or fixed-block reuse dominates (Table 6). On object-aligned public releases and layer-like byte-stable objects (Tables 8 and 9), ReduLink provides authenticated reference substitution at modest overhead over a simple fixed-reference baseline.

The motivating cases are therefore not arbitrary file synchronization. They are stream-serving cases in which the endpoint already knows that a receiver has dictionary state but must still authenticate every reconstruction step and preserve encrypted transport semantics. A CDN edge, registry, backup client, or enterprise service may choose this representation because it avoids exposing plaintext to a WAN optimizer and avoids changing the application protocol into a file-tree delta session, while keeping a stronger per-reference integrity guarantee than a codec-level shared dictionary provides.

Table 26 consolidates the positioning without downgrading RFC 9842's protections. CDT has hashed dictionary identity, HTTPS origin/availability policy, and response discard on decoding failure. ReduLink differs by exposing individually bound chunk references and explicit semantic state repair outside HTTP content coding. The comparison is about granularity and serving abstraction, not about replacing QUIC/TLS integrity.

Table 26. Positioning of ReduLink relative to related redundancy-suppression approaches.

Approach	E2E-encrypted transport	Per-reference auth	Fail-closed repair	Receiver dictionary	Best-fit workload
In-network RE [1-4]	No (needs plaintext vantage)	No	n/a	Network cache	High-redundancy unencrypted links
rsync / LBFS [5]	Endpoints only	Transport integrity only	n/a (delta negotiation)	File/object index	File-tree synchronization
HTTP CDT [19-22]	Yes (HTTPS / HTTP/3)	SHA-256 dictionary identity + TLS	Discard response on decoding failure	Eligible prior HTTP response	HTTP content coding with prior responses
Local compression (gzip/zstd)	Yes	No	n/a	None (intra-object)	Locally repetitive single objects
zstd dictionary delta (--patch-from) [26]	Yes (as payload)	No (codec trust)	No	Exact retained prior stream	Full prior stream retained; best byte savings (Sec. 7.6)
ReduLink (this work)	Yes (QUIC streams)	Yes (per reference)	Yes (MISS then FULL)	Scoped warm dictionary	Warm-state object / layer transfers

13. Limitations and Future Work

The implementation is an application-stream mapping, uses server-only certificate authentication, and substitutes random exporter input because aioquic does not expose live TLS exporter bytes. The public and PyPI objects are transfer models rather than production traces. Userspace shaping is localhost evidence. The committed Linux netem data are concurrent legacy measurements with incomplete environment provenance; they do not support isolated kernel latency or a causal qdisc explanation. A corrected isolated runner is present but has not yet been rerun on Linux. The macOS dummynet path still times out. Bootstrap intervals describe within-run local variability only. Native experiments reach 16 MiB and expose dictionary-budget overflow, but multi-connection, Internet, migration, and larger-than-memory behavior remain out of scope. The formal argument assumes HMAC PRF/random-function behavior for truncation bounds and SHA-256 collision resistance; it is not machine checked. Model framing is optimistic, zstd 1.5.7 dictionary delta dominates where its assumptions hold, and absolute timings are host-dependent.

Future work should integrate actual QUIC TLS exporter output, client/application authentication policy, custom frame negotiation if justified, isolated full-duplex kernel experiments with packet capture and complete provenance, production-style competing flows, larger live traces, broader miss-rate grids, machine-checked analysis, side-channel mitigation, and a true HTTP-level RFC 9842 comparison. The current zstd --patch-from baseline must remain labeled whole-stream dictionary delta, not CDT.

14. Reproducibility

The package contains source, manifests, benchmark runners, result CSV/JSON, figures, citation checks, and tests [15]. Both python3 scripts/run_smoke_validation.py and python3 scripts/run_full_validation.py generate the gitignored deterministic target corpora before checking them, so they run from a clean clone. Full validation executes all tests; it validates committed network evidence but does not rerun privileged kernel experiments or download every external corpus. requirements-lock.txt pins Python packages. The Dockerfile additionally pins zstd 1.5.7 by source checksum and installs GNU rsync, iproute2/tc, and util-linux/taskset. The active builder is scripts/build_manuscript_v3_14.py. Framing JSON records zstd command/version and host provenance; the corrected netem runner records command, platform, Python, aioquic, tc, commit, and active qdisc. The v3.14 files are currently a reviewer-adapted candidate, not yet a public tag; MANUSCRIPT_SHA256.txt binds the local PDF/DOCX, and scripts/verify_public_release.py is intended for use after publication.

15. Conclusion

ReduLink is best understood as authenticated, scoped reference substitution for encrypted endpoint streams. Its byte savings are conditional and are often reproduced or exceeded by simpler fixed-block reuse, compression, or rsync; the contribution is to make reference substitution explicit, individually authenticated, privacy-scoped, repairable, and compatible with native QUIC stream transport, and to position it precisely against both in-network redundancy elimination and HTTP shared-dictionary transport. The evidence supports ReduLink for warm-state object-aligned or layer-like transfers, while negative source-release results show where it should not be used.

Data and Code Availability

All source code, public-corpus manifests, generated corpora, benchmark scripts, result CSV/JSON files, figures, the citation checker, public-release verifier, and the test suite are openly available in the ReduLink repository [15] under the MIT License. Every figure and table is regenerated from the committed result files, so the reported numbers can be reproduced from the artifact. Large external public-release corpora are reconstructed from documented public sources rather than redistributed.

References

[1] N. T. Spring and D. Wetherall, 'A protocol-independent technique for eliminating redundant network traffic,' ACM SIGCOMM, 2000.

[2] A. Anand, V. Sekar, and A. Akella, 'SmartRE: An architecture for coordinated network-wide redundancy elimination,' ACM SIGCOMM, 2009.

[3] A. Anand et al., 'Redundancy in network traffic: Findings and implications,' ACM SIGMETRICS, 2009.

[4] B. Aggarwal et al., 'EndRE: An end-system redundancy elimination service for enterprises,' USENIX NSDI, 2010.

[5] A. Muthitacharoen, B. Chen, and D. Mazieres, 'A low-bandwidth network file system,' ACM SOSP, 2001.

[6] J. Iyengar and M. Thomson, 'QUIC: A UDP-based multiplexed and secure transport,' RFC 9000, IETF, 2021.

[7] M. Thomson and S. Turner, 'Using TLS to secure QUIC,' RFC 9001, IETF, 2021.

[8] J. Iyengar and I. Swett, 'QUIC loss detection and congestion control,' RFC 9002, IETF, 2021.

[9] M. Kuehlewind and B. Trammell, 'Applicability of the QUIC transport protocol,' RFC 9308, IETF, 2022.

[10] B. Trammell et al., 'Manageability of the QUIC transport protocol,' RFC 9312, IETF, 2022.

[11] W. Xia, X. Zou, Y. Zhou, H. Jiang, C. Liu, D. Feng, Y. Hua, Y. Hu, and Y. Zhang, 'The design of fast content-defined chunking for data deduplication based storage systems,' IEEE Transactions on Parallel and Distributed Systems, vol. 31, no. 9, 2020.

[12] M. Gregoriadis, L. Balduf, B. Scheuermann, and J. Pouwelse, 'A thorough investigation of content-defined chunking algorithms for data deduplication,' arXiv:2409.06066, 2024.

[13] B. Alexeev, C. Percival, and Y. X. Zhang, 'Chunking attacks on file backup services using content-defined chunking,' arXiv:2504.02095, 2025.

[14] IEEE 802.3 Ethernet Working Group, 'IEEE 802.3 Ethernet Working Group active projects,' accessed June 2026.

[15] M. Nguyen, 'ReduLink artifact and reproducibility package,' version 3.14 reviewer-adapted candidate, GitHub, 2026. https://github.com/pinkysworld/redulink-deduplex-quic

[16] D. Harnik, B. Pinkas, and A. Shulman-Peleg, 'Side channels in cloud services: Deduplication in cloud storage,' IEEE Security and Privacy, 2010.

[17] M. Bellare, S. Keelveedhi, and T. Ristenpart, 'DupLESS: Server-aided encryption for deduplicated storage,' USENIX Security, 2013.

[18] aioquic project contributors, 'aioquic: QUIC and HTTP/3 implementation in Python,' software repository, version 1.3.0, 2026. https://github.com/aiortc/aioquic

[19] J. Mogul, B. Krishnamurthy, F. Douglis, A. Feldmann, Y. Goland, A. van Hoff, and D. Hellerstein, 'Delta encoding in HTTP,' RFC 3229, IETF, 2002.

[20] D. Korn, J. MacDonald, J. Mogul, and K. Vo, 'The VCDIFF generic differencing and compression data format,' RFC 3284, IETF, 2002.

[21] J. Butler, W.-H. Lee, B. McQuade, and K. Mixter, 'A proposal for shared dictionary compression over HTTP (SDCH),' IETF Internet-Draft draft-lee-sdch-spec-00, 2016.

[22] P. Meenan and Y. Weiss, 'Compression dictionary transport,' RFC 9842, IETF, 2025.

[23] H. Krawczyk, M. Bellare, and R. Canetti, 'HMAC: Keyed-hashing for message authentication,' RFC 2104, IETF, 1997.

[24] H. Krawczyk and P. Eronen, 'HMAC-based extract-and-expand key derivation function (HKDF),' RFC 5869, IETF, 2010.

[25] R. Jain, D.-M. Chiu, and W. Hawe, 'A quantitative measure of fairness and discrimination for shared computer systems,' DEC Research Report TR-301, 1984.

[26] Y. Collet and M. Kucherawy, 'Zstandard compression and the application/zstd media type,' RFC 8878, IETF, 2021.